



# **Penetration Testing Report and Incident Response**

**for**

**Bodgeit Store Website**

**BY: Sensei G-Technology**

**Table of Contents**

|                                                    |           |
|----------------------------------------------------|-----------|
| <b>Executive Summary</b>                           | <b>1</b>  |
| Summary of Results                                 | 1         |
| <b>Attack Narrative</b>                            | <b>1</b>  |
| <b>Technical Flndings</b>                          | <b>3</b>  |
| Commented Admin Page in Source Code                | 3         |
| Login Bypass via SQL injection                     | 5         |
| Broken Access Control (POST to GET Password Reset) | 6         |
| Logic Flaw Through Integer Overflow                | 9         |
| <b>Incident Response Plan</b>                      | <b>11</b> |
| Business Case for SOC Solution                     | 11        |
| Draft out NIST Process and Supporting Policies     | 12        |
| Resources Required                                 | 12        |
| <b>References</b>                                  | <b>13</b> |
| <b>Bibliography</b>                                | <b>13</b> |
| <b>Appendix</b>                                    | <b>13</b> |
| <b>Conclusion</b>                                  | <b>14</b> |

Left Blank



## 1. Executive Summary

Bodgeit Store website was subjected to a comprehensive penetration test, where Cross-Site Scripting (XSS) vulnerabilities and the use of SQLmap tool were out of scope. I was able to identify multiple vulnerabilities that could be exploited by an attacker to compromise the website confidentiality, integrity, and availability. This report provides a detailed technical breakdown of the vulnerabilities, their exploitation using the MITRE ATT&CK framework, and recommendations for mitigating these risks. Also it includes an incident response plan to address potential breaches, supported by a business case for implementing a Security Operations Center (SOC) solution.

## 2. Summary of Results

Initial reconnaissance of the Bodgeit store web application resulted in the discovery of misconfigured directory access to the admin page without authentication which grant access to rich information of other users. An attempt to login to one of the user account via login page was successful through bypassing via SQL injection, then was able to access the user account and changed password for that specific user by changing the request method resulting in total account takeover through broken access control. Finally I further access the user basket and was able to make the website owe us money via integer overflow.

## 3. Attack Narrative

For this test I was given access to the virtual machine which contain several web application, the focus is to pen-test the bodgeit store web application for vulnerabilities. The intent was to closely simulate an attack on the website without internal information

In an attempt to identify the target website I setup a the target virtual machine and the attacking machine which was Kali linux on a VMware using Host only network and performed a network discovery on the attacking machine(*kali*) with the *netdiscover* command which allowed us to view devices on our network. There I found the target machine IP address which was **192.168.108.128**.

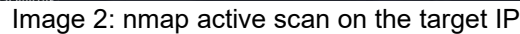


Image 3: nmap result show Open port 80 and 443



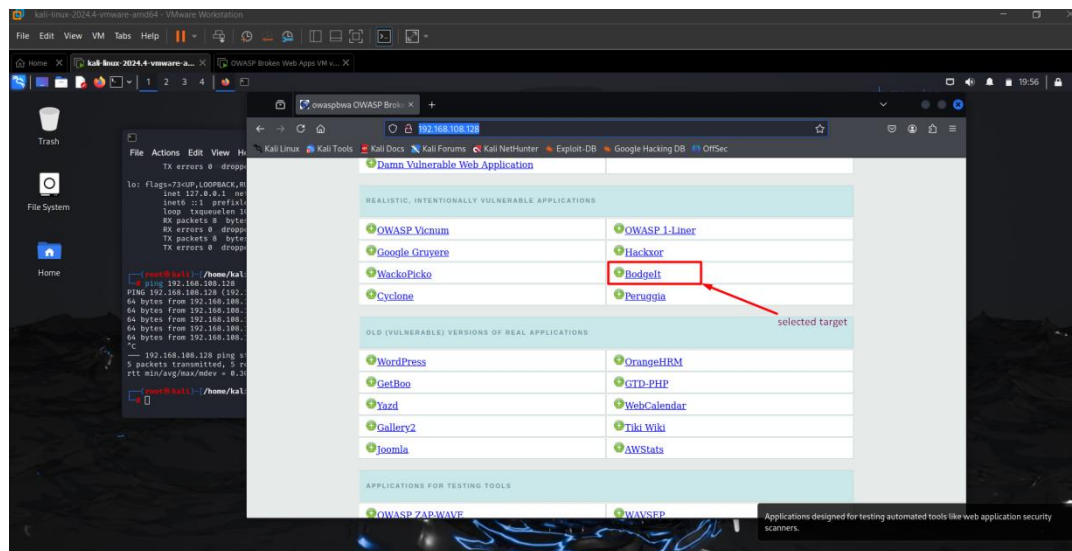


Image 4: Selected Bodgeit

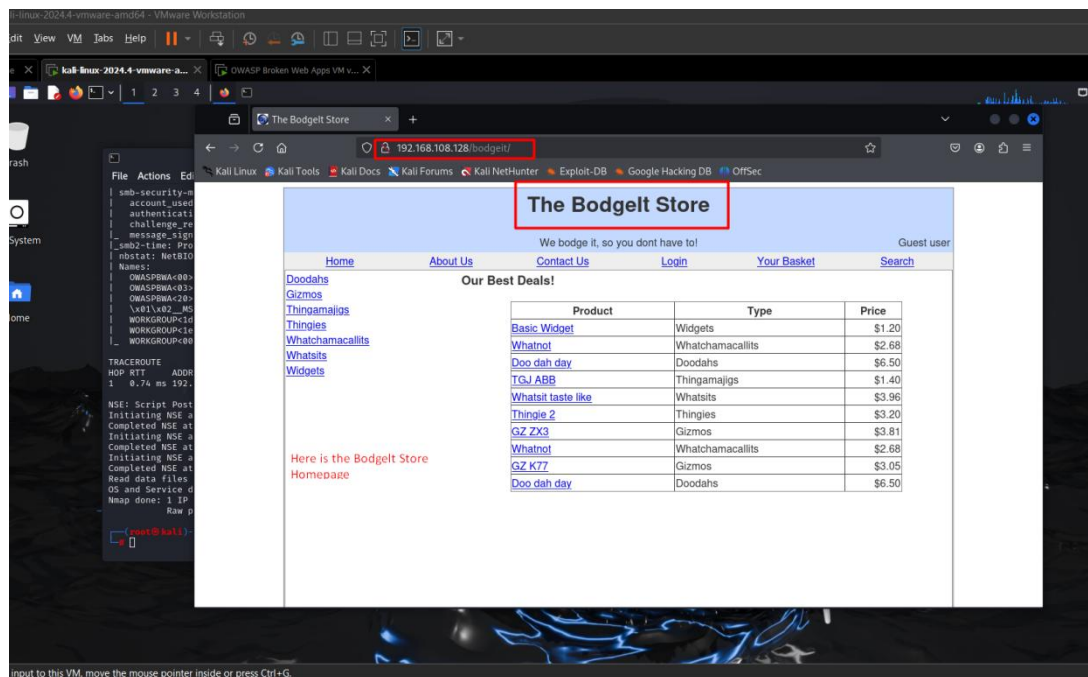


Image 5: Bodgeit Store Homepage

## 4. Technical Findings

### 4.1. Vulnerability 1: Commented Admin Page in Source Code

#### 4.1.1. MITRE ATT&CK Matrix Mapping

- Tactic: Discovery and Initial Access
- Technique: T1083 - (File and Directory Discovery) and T1190 - (Exploit Public-Facing Application)

#### 4.1.2. Risk Matrix

Table 1: Risk Matrix

| Likelihood | Impact | Risk Level |
|------------|--------|------------|
| High       | High   | Critical   |

4.1.3. Technical Summary

I inspected the Bodgeit website source code by hitting *ctrl U* on the keyboard, where I discovered the commented-out admin page e.g `<!-- <a href="/admin">Admin Panel</a> -->`. I attempt visiting the page like this: `/admin` and was able to access the admin page without any authentication.

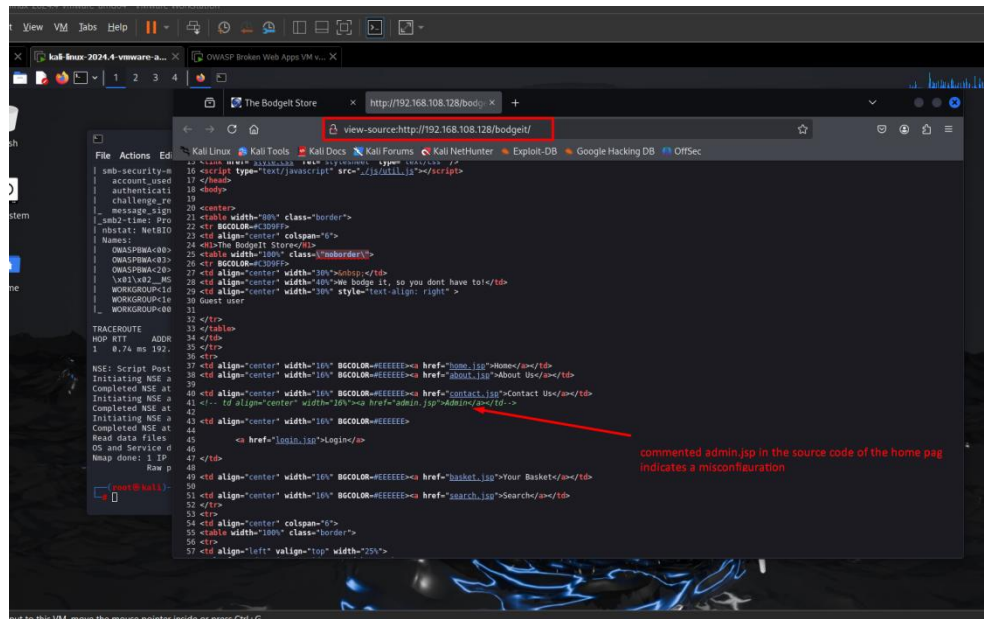


Image 6: Commented Admin page

This provided initial access to the website where I was able to view other users details such as emails, user id, basket id, and roles.

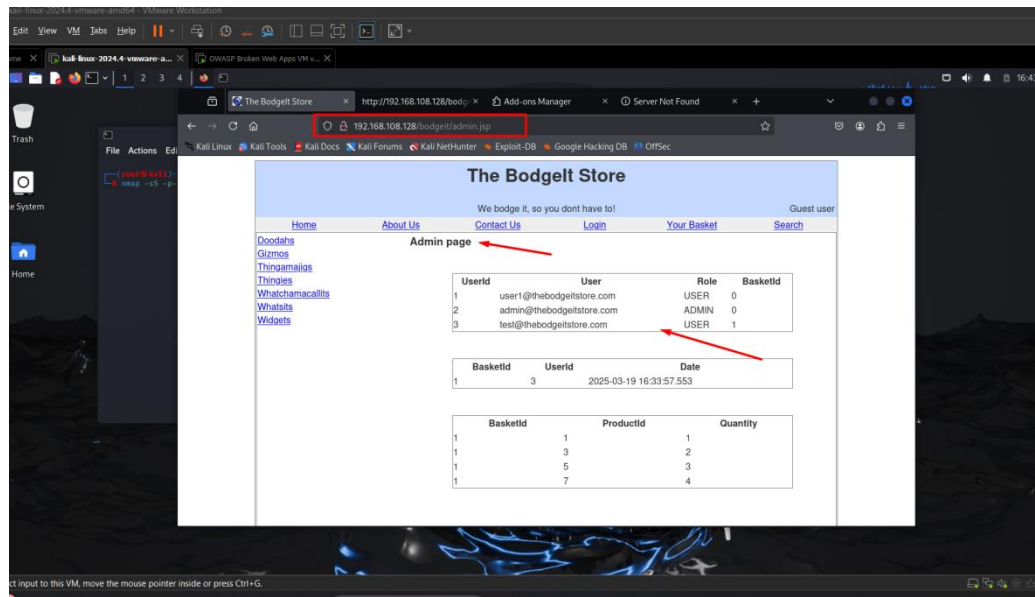


Image 7: Admin page accessible without authentication, showing other user details and ids

#### 4.1.4. Recommendations (MITRE ATT&CK Framework)

##### Code Review and Static Analysis (T1083)

- It is important to perform regular code reviews and to identify and remove unintended commented-out code files and directories in the source code
- Implement static analysis tools like ESLint to automatically detect and flag commented-out sensitive information or endpoints in the source code

##### Access Control Implementation (T1190)

- Ensure admin pages or endpoints are restricted using robust authentication and authorization even if they are not linked.
- Use role-based access control to restrict access to admin pages to only authorized users.

#### 4.2. Vulnerability 2: Login Bypass via SQL injection

##### 4.2.1. MITRE ATT&CK Matrix Mapping

- Tactic: Defense Evasion and Initial Access.
- Technique: T1059 - (Command and Scripting Interpreter), T1190 - (Exploit Public-Facing Application) and T1565 - (Data Manipulation).

##### 4.2.2. Risk Matrix

Table 2: Risk Matrix

| Likelihood | Impact | Risk Level |
|------------|--------|------------|
| High       | High   | Critical   |

##### 4.2.3. Technical Summary

Having discovered various user accounts in the admin page, I head to the login page to try out some of the account. Since I did not have a password, I thought of a bypass method using SQLi, where I used input in to the username as the email of one of the users e.g. [user1@thebodgeitstore.com](mailto:user1@thebodgeitstore.com) and password as ' OR '1'='1.



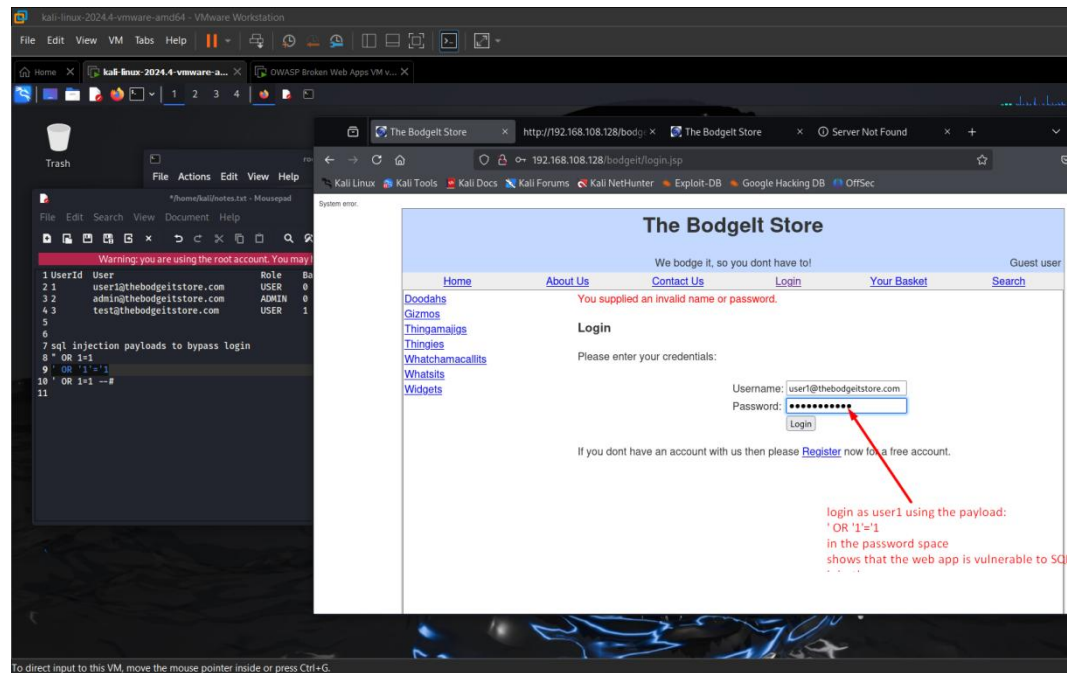


Image 8: SQL injection in login form

This attempt grant me access into the user account indicating SQL injection and lack of input validation and sanitization causing a login bypass.

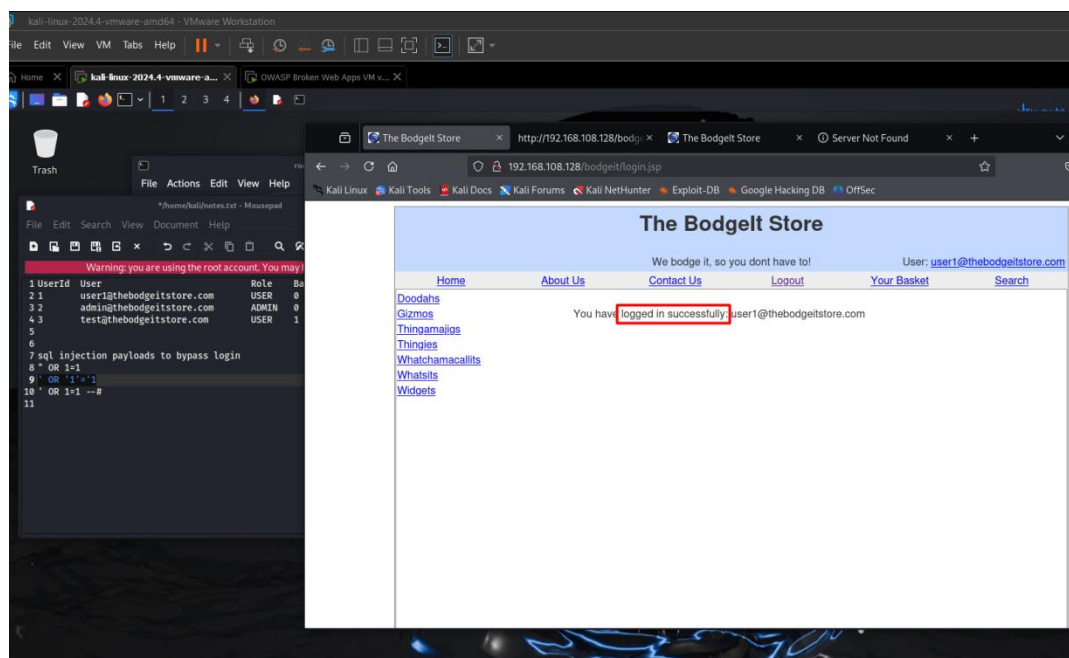


Image 9: Login bypass successful through SQLi

#### 4.2.4. Recommendations (MITRE ATT&CK Framework)

##### Input Validation (T1190)

- Use strict input validation on all user inputs.
- Use whitelisting to allow only valid characters and patterns in user inputs.

**Defense Evasion (T1565 - Data Manipulation)**

- Implement proper error handling, logging, and conduct regular security testing for SQLi vulnerabilities (T1565).

**Defense Evasion (T1059 - Command and Scripting Interpreter)**

- Conduct code reviews and analysis to detect insecure coding practices
- Train developers on secure coding techniques to prevent common vulnerabilities like SQLi.

**4.3. Vulnerability 3: Broken Access Control (POST to GET Password Reset)****4.3.1. MITRE ATT&CK Matrix Mapping**

- Tactic: Privilege Escalation
- Technique: T1220 - (Abuse Elevation Control Mechanism)

**4.3.2. Risk Matrix**

Table 3: Risk Matrix

| Likelihood | Impact | Risk Level |
|------------|--------|------------|
| High       | High   | Critical   |

**4.3.3. Technical Summary**

Here, after gaining access to the user account I clicked the email displayed on the account which redirects me to password reset page, I used the browser developer tool to inspect the form and I noticed it uses a *POST* request method then changed it to *GET* request and hit enter. This expose critical vulnerability allowing attacker to change HTTP request to trigger password reset for unauthorized accounts.

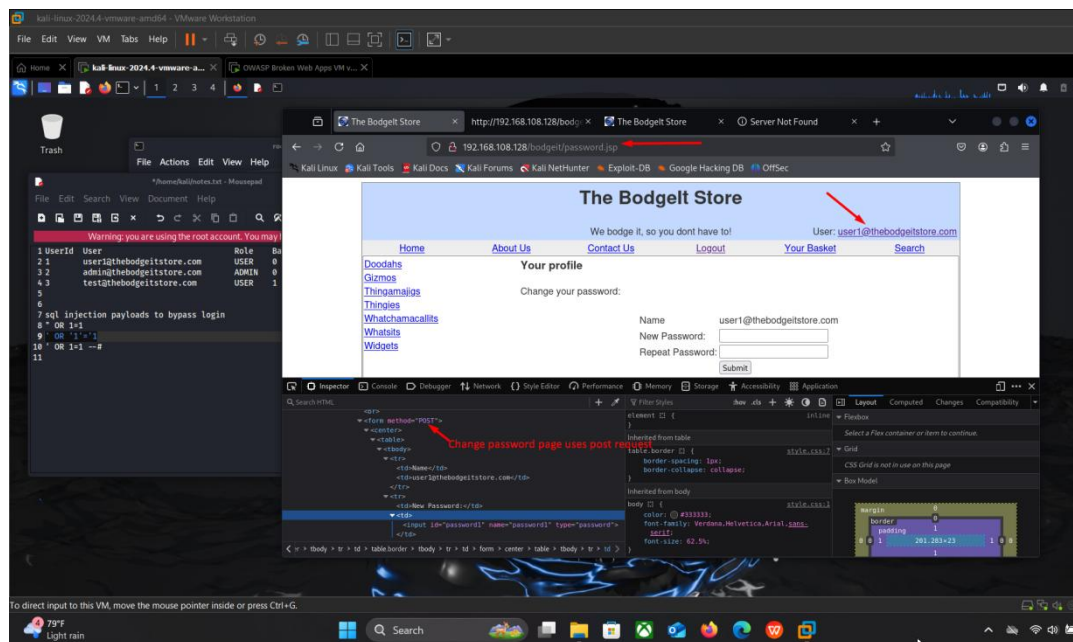


Image 10: Change password page uses post request

After changing the request method I attempt changing the user1 password to *password* and the attempt was successful as seen in the screenshot below.

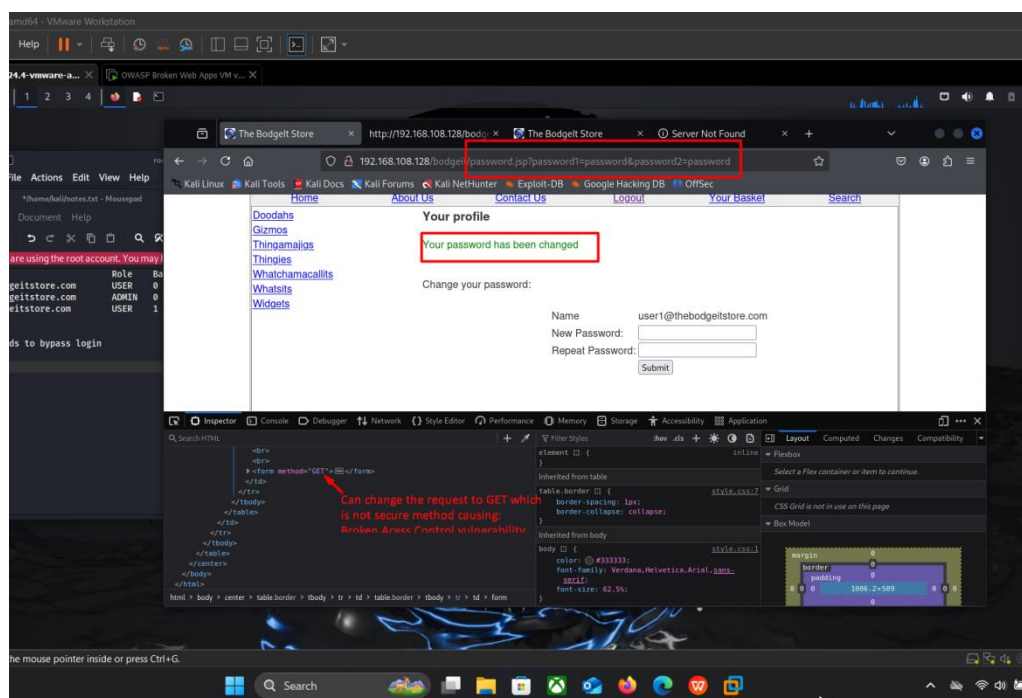


Image 11: Broken access control where Post can be changed to GET request in change password page

#### 4.3.4. Recommendations (MITRE ATT&CK Framework)

##### Strict Access Controls (T1220)

- Validate user roles and permissions for all password resets.

- Enforce that only authenticated and authorized users can initiate password reset requests.

### Use and Validates One Time Password (OTP) Token (T1220)

- Enforce secure, time-limited OTP for password reset functionality. Verify the token's validity before processing the request.

## 4.4. Vulnerability 4: Logic Flaw Through Integer Overflow

### 4.4.1. MITRE ATT&CK Matrix Mapping

- Tactic: Impact and Defence Evasion
- Technique: T1485 - (Data Destruction) and T1565 - (Data Manipulation)

### 4.4.2. Risk Matrix

Table 4: Risk Matrix

| Likelihood | Impact | Risk Level |
|------------|--------|------------|
| Medium     | High   | High       |

### 4.4.3. Technical Summary

After changing the password, I moved on to checking the basket functionality by clicking the *Your Basket* link on the page and passing the request through the Burp Suite proxy tool to view the request sent. Burpsuit can be used for analyzing and manipulating http requests.

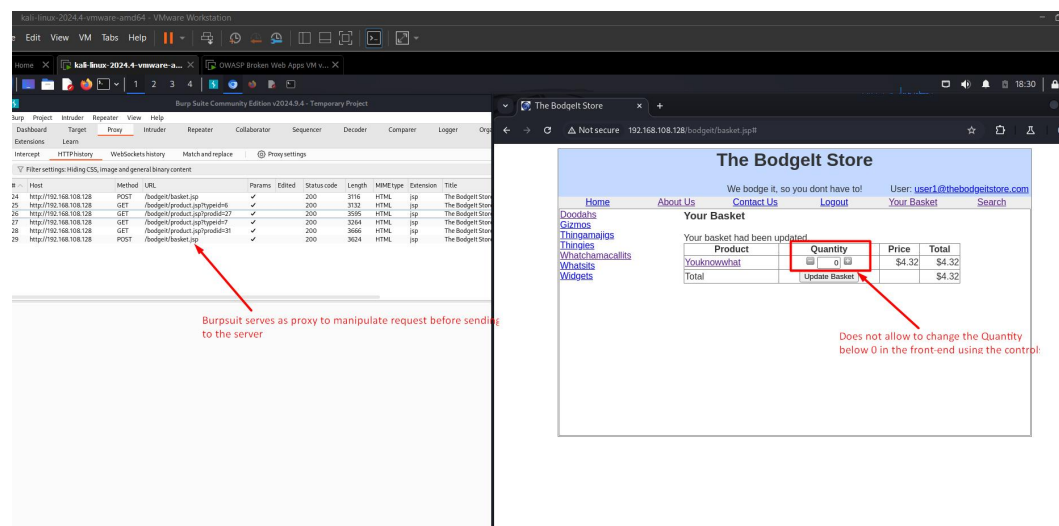


Image 12: Burp suit as proxy

In the Basket page there is a table containing *Product*, *Quantity*, *Prize* and *Total*, with a button that allows us to update the basket. Also there is are buttons that allows us to increase or decrease quantity of products selected. I tried ti try this functionality and I noticed that the decrease functionality stops at 0 not allowing negative numbers so I decided to check if this can be

manipulated through Burpsuite. I increase the quantity and clicked the *Update Basket* button but intercepted the request in Burpsuite.

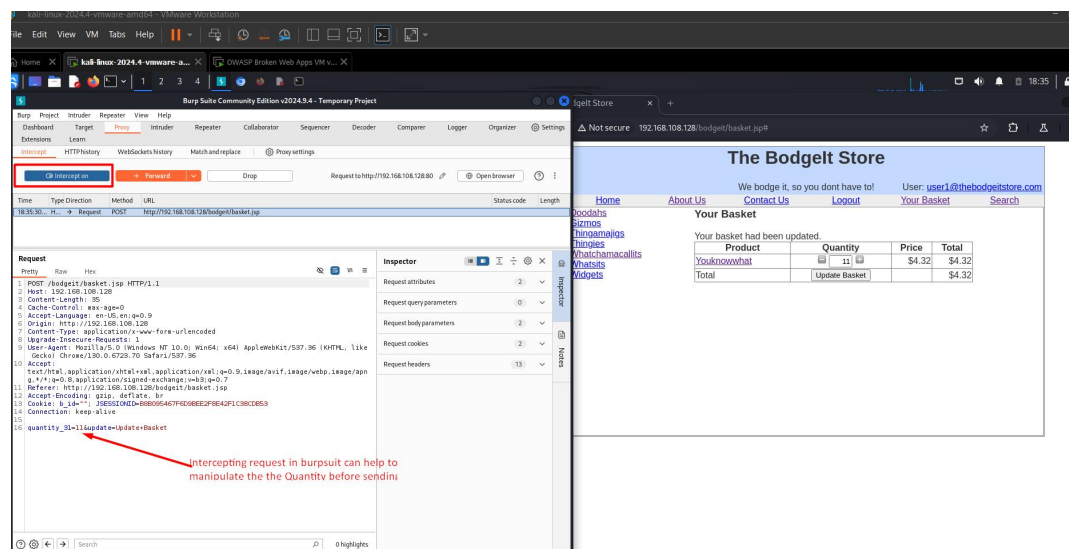


Image 13: Intercepting request to manipulate Quantity

I was able to capture the request then change the quantity to a negative number e.g -11, then send the request using forward button available in Burpsuite. This attempt to change the quantity of product was successful with the basket updated message on the website making the store to owe us money.

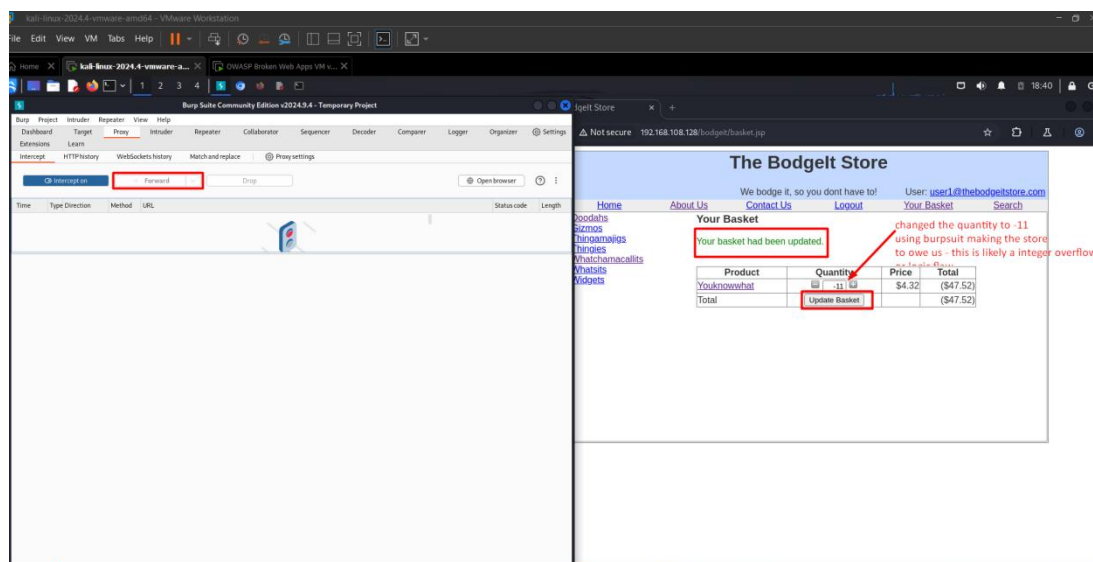


Image 14: Integer overflow or logic flaw vulnerability

This shouldn't be allowed on website as it indicates a logic flaw via integer overflow vulnerability and could disrupt business financial data or transaction logs may be corrupted impacting integrity.

#### 4.4.4. Recommendations (MITRE ATT&CK Framework)

##### Use Safe Programming Practices(T1565)

- Use programming languages or libraries that handle integer overflows safely (e.g., Rust, Python).
- Implement checks to detect and handle overflows in calculation

##### Data Integrity Checks(T1485)

- Implement checksum or hashing to verify the integrity of financial data and transaction logs.

##### Monitoring and Alerts

- Set up monitoring and alerts for unusual patterns in financial transactions or balances.

## 5. Incident Response Plan

The mentioned vulnerabilities pose significant risks to the organization, including data breaches, reputation damage, and financial losses. Immediate remediation is required to mitigate these risks. Additionally, implementing a proactive SOC solution will help the organization's ability to detect and respond to future threats.

### 5.1. Business Case for SOC Solution

A Security Operation Center (SOC) will provide real-time monitoring, threat detection, and incident response capabilities, reducing the time find and respond to security incidents. This minimizes potential damages and ensure compliance with regulatory requirements safeguarding organization's reputation and baseline.

#### 5.1.1. Recommended SOC Solution

##### Solution Choice

- **Managed SOC Service (e.g., IBM QRadar, Splunk SOAR)**

##### Justification

- **Scalability:** Managed SOC services scale with organizational growth level or and development.
- **Expertise:** Provides access to skilled cybersecurity professionals.
- **Cost-Effectiveness:** Eliminates the need for extensive in-house infrastructure and staffing.

## 5.2. Draft out NIST Process and Supporting Policies

### 5.2.1. NIST Process

1. **Identify:** Identify, record assets and define roles and responsibilities.
2. **Protect:** Implement security controls on identified assets or in the network (e.g., firewalls, encryption).
3. **Detect:** Deploy monitoring and detection tools to identify anomalies in the network.
4. **Respond:** Establish an incident response team, playbook and policies.
5. **Recover:** Develop recovery plans such as regular backups and conduct post-incident reviews like testing.

### 5.2.2. Policies

1. **Access Control Policies:** Enforce least privilege principles on access controls.
2. **Incident Response Policy:** Define procedures for reporting and handling incident such as regular pen-testing, testing backups and effective monitoring of assets.
3. **Data Protection Policy:** Mandate encryption and secure storage of sensitive data (e.g., robust authentication, authorization, use of secure ports).

## 5.3. Resources Required

### 5.3.1. Human Resources

- Security Analysts
- Incident Response Team
- Compliance Officer

### 5.3.2. Technology Resources

- SIEM Tool (e.g., Splunk, QRadar, Wazuh)
- Endpoint Detection and Response (EDR) Solution (e.g., Snort, Microsoft Defender, CrowdStrike Falcon)
- Vulnerability Scanning Tools (e.g., Nikto, Nessus)

### 5.3.3. Financial Resources

- Budget for SOC subscription
- Training and certification costs for staff
- Infrastructure upgrades (if needed or when neccessary)



## 6. References

- MITRE ATT&CK Framework. Retrieved from <https://attack.mitre.org>
- National Institute of Standards and Technology (NIST). "Computer Security Incident Handling Guide (SP 800-61)." Retrieved from <https://nvlpubs.nist.gov>
- OWASP Top Ten Web Application Security Risks. Retrieved from <https://owasp.org>
- SANS Institute. "Critical Security Controls." Retrieved from <https://www.sans.org>

## 7. Bibliography

### 1. Academic Papers:

- Paul Cichonski et al. (2012). "Computer Security Incident Handling Guide." NIST Special Publication 800-61, Revision 2.
- Howard, M., & LeBlanc, D. (2002). "Writing Secure Code" (2nd ed.). Microsoft Press.

### 2. Online Articles :

- OWASP Foundation. (2023). "Top Ten Web Application Security Risks." Retrieved from <https://owasp.org>.
- SANS Institute. (2023). "What is a Security Operations Center (SOC)?" Retrieved from <https://www.sans.org>.

## 8. Appendix

### 8.1. Scripts and Commands

- netdiscover: to discover devices on the network
- nmap -sS -p- -Pn -T4 -v -O: To discover open ports, services and their versions
- Ctrl U: Command for viewing page source
- ' OR '1'='1: SQL injection payload to bypass login

### 8.2. Risk Matrix Template

Table 5: Risk Matrix Template

| Likelihood ↓ | Impact → | Low      | Medium | High     |
|--------------|----------|----------|--------|----------|
| Low          |          | Low Risk | Medium | Medium   |
| Medium       |          | Medium   | Medium | High     |
| High         |          | Medium   | High   | Critical |

### 8.3. Tools Used During Penetration Test

- Nmap: For port scanning
- Burp Suite: For intercepting and manipulating HTTP requests



## 9. Conclusion

This report highlights critical vulnerabilities in the Bodgeit Store website and provides actionable recommendations aligned with the MITRE ATT&CK framework. The proposed SOC solution, supported by a robust NIST-based process and policies, will significantly enhance the organization's security posture. Immediate action is advised to mitigate risks and protect sensitive data.

**9.1. Final Recommendation:** Implement the SOC solution and remediate vulnerabilities promptly to safeguard the organization's assets and reputation.